Analysis of Numerical Algorithms

From Root-Finding to Fractals

Author: Jaya Patel

Institution: IIT Delhi

Department: Mathematics and Computing

Abstract

Numerical methods form the computational backbone of modern science and engineering. This paper presents an analysis of classical numerical algorithms root finding methods (Bisection and Secant), polynomial interpolation techniques (Lagrange, Newton Divided Differences, and Cubic Hermite), and Newton's Method for nonlinear equations. Each algorithm is accompanied by its mathematical derivation and a Python implementation. The paper culminates in a visually striking application: Newton Fractals where the geometry of complex roots gives rise to infinite, self-similar beauty at the boundary of convergence basins. We also outline directions for future work, including dynamic, interactive fractal visualizations.

1 Introduction to Numerical Algorithms

While pure mathematics focuses on exact analytical solutions, practical applications in physics, engineering, and finance frequently involve equations without closed form answers. Non-linear systems and complex differential equations often challenge standard algebraic manipulation. Numerical methods solve these complex equations by breaking them down into step by step calculations that gradually converge on a close answer.

A numerical algorithm is a finite, deterministic procedure designed to compute a solution within a specified error tolerance. Numerical methods focus on what is actually efficient to compute. They use approximation to find answers for equations that are otherwise impossible to solve by hand.

1.1 Motivation

This project is motivated by the geometric properties of Newton's method when applied to multi variable root finding problems. Because the algorithm relies heavily on initial guesses, slight variations in the starting point can alter which root the system converges to, or whether it converges at all. These fractals plot the boundaries where convergence shifts, showing how a formula can produce highly unpredictable results. Studying these dynamics gives a clear picture of how stable or predictable an iterative algorithm actually is.

2 Root-Finding Methods

Finding the points where an equation equals zero expressed as $f(x) = 0$ is a fundamental problem in applied mathematics. Because exact analytical solutions are rarely available for most practical equations, we rely on iterative methods. These numerical techniques start with one or more initial guesses x_0 and step-by-step approximate the true root α within a desired level of accuracy.

2.1 The Bisection Method

The bisection method is a reliable root-finding technique based on the Intermediate Value Theorem. Assume that a function $f(x)$ is continuous on a given interval $[a, b]$ and satisfies the condition:

$$f(a)f(b) < 0 \tag{1}$$

This condition ensures that $f(a)$ and $f(b)$ have opposite signs, meaning the function must cross the x-axis at least once within the interval. Consequently, $f(x)$ is guaranteed to have at least one root α in $[a, b]$. While the interval $[a, b]$ is typically chosen to isolate a single unique root, any iterative algorithm applying the bisection method under these conditions will always converge to a root α within the interval.

2.1.1 Python Implementation

```python
def f(x):
    return x**2 - 4*x - 10

def bisection_method(f, a, b, tol=1e-6, max_iter=100):
    if f(a) * f(b) >= 0:
        raise ValueError("The Bisection Method requires that "
                         "f(a) and f(b) have opposite signs.")
    for _ in range(max_iter):
        c = (a + b) / 2
        fc = f(c)

        if abs(fc) < tol or (b - a) / 2 < tol:
            return c

        if f(a) * fc < 0:
            b = c
        else:
            a = c
    return c

root = bisection_method(f, -2.0, 0.0, tol=1e-4)
print(f"Root: {root}")
```

Listing 1: Bisection Method

The Secant Method

Similar to Newton's method, the secant method approximates the graph of $y = f(x)$ using a straight line near the true root α. This method requires two initial root estimates, x_0 and x_1. By approximating the function's curve with a secant line passing through the points $(x_0, f(x_0))$ and $(x_1, f(x_1))$, we find where this line intersects the x-axis. This intersection point, denoted as x_2, serves as an updated approximation of α.

Using the slope formula for the secant line, we equate the slope between the two known points to the slope leading to the root estimate x_2:

$$\frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{f(x_1) - 0}{x_1 - x_2} \tag{2}$$

Solving this equation for x_2 yields the iterative formula:

$$x_2 = x_1 - f(x_1) \cdot \frac{x_1 - x_0}{f(x_1) - f(x_0)} \tag{3}$$

By repeating this iterative process with successive pairs of approximations, we can generalize the procedure. The general recurrence relation for the secant method is defined as:

$$x_{n+1} = x_n - f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}, \quad n \geq 1 \tag{4}$$

The secant method is not globally guaranteed to converge. However, when it does converge, its rate of convergence is typically faster than that of the bisection method.

2.1.2 Python Implementation

```python
def secant(f, x0, x1, eps, itmax):
    itnum = 1

    while True:
        x2 = x1 - (f(x1) *(x1-x0))/(f(x1)-f(x0))

        if abs(x2 - x1) <= eps:
            return x2, 0

        if itnum == itmax:
            return x2, 1
        itnum += 1
        x0 = x1
        x1=x2

f  = lambda x: x**6-x- 1

root, ier = secant(f,x0=1,x1=2.0,eps=1e-7, itmax=100)

if ier == 0:
    f_root=f(root)
    print(f"Converged! Root = {root},{f_root}")
elif ier == 1:
    print(f"Max iterations reached. Best estimate = {root}")
```

Listing 2: Secant Method

3 Polynomial Interpolation

Let $x_0, x_1, \ldots, x_n$ be distinct real or complex numbers, and let $y_0, y_1, \ldots, y_n$ be their associated function values. The problem of polynomial interpolation involves finding a polynomial $p(x)$ that precisely matches this given data at the specified nodes:

$$p(x_i) = y_i, \quad i = 0, 1, \ldots, n \tag{5}$$

By expressing a general polynomial of degree m as:

$$p(x) = a_0 + a_1 x + \cdots + a_m x^m \tag{6}$$

we observe that the polynomial contains $m+1$ independent coefficients to be determined. Interpolation can be used in stock market to estimate missing asset prices or calculate asset values at specific time intervals when direct exchange data is unavailable. It is particularly valuable in fixed-income markets for constructing continuous yield curves from a limited set of observed bond maturities.

3.1 Lagrange Interpolation

Lagrange interpolation builds the polynomial from basis functions, each equal to 1 at one node and 0 at all others.

Lagrange basis polynomial:

$$L_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^{n} \frac{x - x_j}{x_k - x_j}$$

Interpolating polynomial:

$$P(x) = \sum_{k=0}^{n} y_k \, L_k(x)$$

Python Implementation:

```python
def lagrange_interpolation(x, y, t):
    n = len(x)
    p= 0.0
    for i in range(n):
        L = 1.0
        for j in range(n):
            if i != j:
                L *= (t - x[j]) / (x[i] - x[j])
        p += y[i] * L
    return p
#example:
t=[2,3,8]
x=[1,3,5,7,8]
y=[2,1,5,3,9]
for j in t:
    y2=lagrange_interpolation(x, y, j)
    print(f"value at {j} = {y2}")
```

Listing 3: Lagrange Interpolation

Figure 1 shows Lagrange interpolation applied to $f(x) = \sin(x)$ over $[0, \pi]$ using three random nodes. Even with just a few points, the interpolating polynomial $P(x)$ fits the true sine curve remarkably well, showing how effective Lagrange interpolation is at reconstructing smooth functions from limited data.

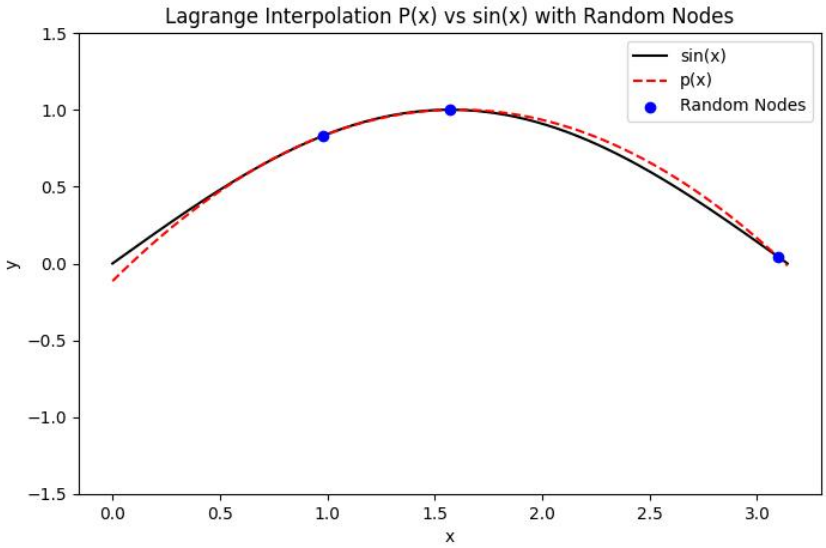

Figure 1: Lagrange interpolation $P(x)$ approximating $\sin(x)$ with 3 random nodes.

3.2 Newton's Divided Difference Interpolation

The Lagrange form of the interpolation polynomial can be used for interpolation to a function given in tabular form

$$p_n(x) = p_{n-1} + C(x) \quad \text{where } C(x) = \text{correction term} \tag{7}$$

Then, in general, $C(x)$ is a polynomial of degree n, since usually $\text{degree}(p_{n-1}) = n-1$ and $\text{degree}(p_n) = n$. Also we have

$$C(x_i) = p_n(x_i) - p_{n-1}(x_i) = f(x_i) - f(x_i) = 0 \quad i = 0, \ldots, n-1 \tag{8}$$

Thus

$$C(x) = a_n(x - x_0) \cdots (x - x_{n-1}) \tag{9}$$

Since $p_n(x_n) = f(x_n)$, from that

$$a_n = \frac{f(x_n) - p_{n-1}(x_n)}{(x_n - x_0) \cdots (x_n - x_{n-1})} \tag{10}$$

For reasons derived below, this coefficient a_n is called the nth-order Newton divided difference of f, and it is denoted by

$$a_n \equiv f[x_0, x_1, \ldots, x_n] \tag{11}$$

Thus our interpolation formula becomes

$$p_n(x) = p_{n-1}(x) + (x - x_0) \cdots (x - x_{n-1}) f[x_0, \ldots, x_n] \tag{12}$$

Python Implementation

```python
def divdif(f_values, x):
    d = f_values
    n = len(x) - 1
    for i in range(1, n + 1):
        for j in range(n, i - 1, -1):
            d[j] = (d[j] - d[j-1]) / (x[j] - x[j-i])
    return d

def interp(d, x, t):
    n = len(d) - 1
    p = d[n]
    for i in range(n - 1, -1, -1):
        p = d[i] + (t - x[i]) * p
    return p

x = [0, 1, 2, 3]
f = [1, 2, 5, 10]

d = divdif(f, x)
t = 1.5
print(interp(d, x, t))
```

Listing 4: Newton's Divided Difference Interpolation

3.3 Hermite Interpolation

Hermite interpolation looks for an interpolating polynomial $p(x)$ that matches both the function values and the first derivative values of a given function $f(x)$ at a specified set of nodes. This technique is widely used as a mathematical tool for developing Gaussian numerical integration schemes and creating numerical methods to solve differential equations.

Given n distinct nodes $x_1, \ldots, x_n$ along with function values y_i and derivative values y'_i, the problem imposes $2n$ distinct conditions:

$$p(x_i) = y_i, \quad p'(x_i) = y'_i, \quad i = 1, \ldots, n \tag{13}$$

Because there are $2n$ total constraints, we seek an interpolating polynomial $H_n(x)$ of degree at most $2n - 1$. Using a generalized base of basis polynomials $h_i(x)$ and $\bar{h}_i(x)$ related to the Lagrange structure $l_i(x)$, the unique Hermite interpolating polynomial is explicitly constructed as:

$$H_n(x) = \sum_{i=1}^{n} y_i h_i(x) + \sum_{i=1}^{n} y'_i \bar{h}_i(x) \tag{14}$$

Python Implementation

```
def l(i, x, t):
    ans = 1
    for j in range(len(x)):
        if i != j:
            ans *= (t - x[j]) / (x[i] - x[j])
    return ans
def l1(i,x,t):
    deno=1
    for j in range(len(x)):
        if i!=j:
            deno *= (x[i] - x[j])
    num=0
    for k in range(len(x)):
        if k != i:
            ans=1
            for j in range(len(x)):
                if j!=i and j!=k:
                    ans*=t-x[j]
            num+=ans
    f=num/deno
    return f
def hermite(y1,y,x,t):
    s=0
    for i in range(len(x)):
        hi=(1-2*(l1(i,x,x[i]))*(t-x[i]))*((l(i,x,t))**2)
        hi1=(t-x[i])*((l(i,x,t))**2)
        s+=((y[i])*hi + (y1[i])*hi1)
    return s
x=[0,1,2,3]
y=[0,1,0,0]
y1=[0,0,0,0]
print(hermite(y1,y,x,2.5))
```

Listing 5: Hermite Interpolation

3.4 Cubic Hermite Interpolation

Hermite basis functions on $[x_i, x_{i+1}]$, with $t = \frac{x-x_i}{x_{i+1}-x_i}$:

$$h_{00}(t) = 2t^3 - 3t^2 + 1, \quad h_{10}(t) = t^3 - 2t^2 + t$$
$$h_{01}(t) = -2t^3 + 3t^2, \quad h_{11}(t) = t^3 - t^2$$

Interpolant:

$$p(t) = h_{00}f(x_i) + h_{10}(x_{i+1} - x_i)f'(x_i) + h_{01}f(x_{i+1}) + h_{11}(x_{i+1} - x_i)f'(x_{i+1})$$

Python Implementation

```
def y(x):
    y=[]
    for i in x:
        f=1/(1+(i*i))
        y.append(f)

    return y

def s_i(x,k):
    i=x[k]
    f=-(2*i)/((1+i*i)**2)
    return f

def find_interval(x, t):
    for k in range(len(x) - 1):
        if x[k] <= t <= x[k+1]:
            return k
    return -1

def P(x,y,s_i,t,k):
    c1=y[k]
    c2=s_i(x,k)
    f_xi_xi1 = (y[k]-y[k+1])/(x[k]-x[k+1])
    del_xi=x[k+1]-x[k]
    c4 = (c2 + s_i(x, k+1) - 2 * f_xi_xi1) / (del_xi * del_xi)
    c3 = ((f_xi_xi1 - c2) / del_xi) - (c4 * del_xi)
    dx = t - x[k]
    p = c1 + c2 * dx + c3 * (dx ** 2) + c4 * (dx ** 3)
    return p
x=[1,3,6,8]
t=4
y=y(x)
k=find_interval(x,t)
ans=P(x,y,s_i,t,k)
print(ans)
```

Listing 6: Cubic Hermite Interpolation

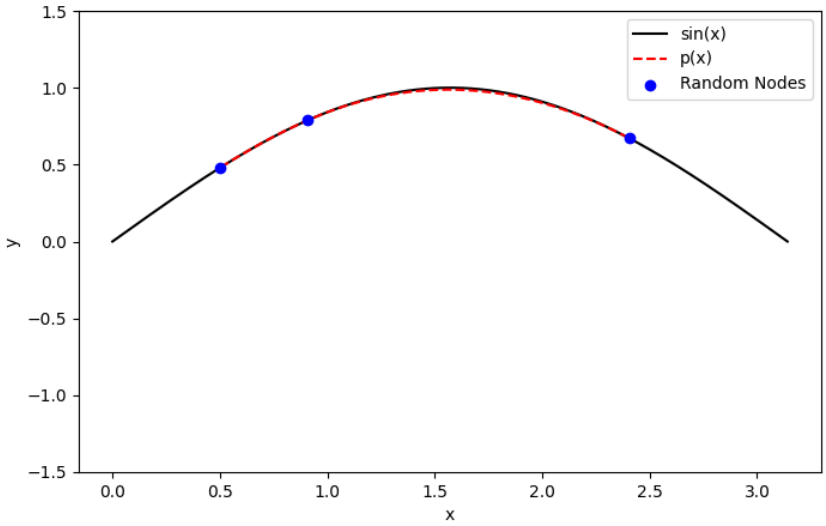

Figure 2: Cubic Hermite Interpolation $P(x)$ approximating $\sin(x)$ with 3 random nodes.

Figure 2 shows Cubic Hermite interpolation applied to $f(x) = \sin(x)$ over $[0, \pi]$ using three random nodes. Even with very few data points, the interpolating polynomial $P(x)$ fits the true sine curve closely, showing how effectively Cubic Hermite interpolation reconstructs smooth, continuous functions from limited samples.

4 Newton's Method

Newton's method is a widely used iterative technique for finding the roots of a nonlinear equation $f(x) = 0$. Given an initial approximation x_0 close to the true root α, the method approximates the graph of $y = f(x)$ by its tangent line at $(x_0, f(x_0))$. The intersection of this tangent line with the x-axis provides an updated root approximation, x_1. Repeating this process geometric step-by-step yields a sequence of iterates $\{x_n\}$ via the standard formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad n \geq 0 \tag{15}$$

An alternative derivation expands $f(x)$ using a Taylor series about x_n:

$$f(x) = f(x_n) + (x - x_n)f'(x_n) + \frac{(x - x_n)^2}{2}f''(\xi) \tag{16}$$

Evaluating this at the true root $x = \alpha$ where $f(\alpha) = 0$ and dropping the higher-order error term isolates the convergence error expression:

$$\alpha - x_{n+1} = -(\alpha - x_n)^2 \frac{f''(\xi_n)}{2f'(x_n)}, \quad n \geq 0 \tag{17}$$

This quadratic relationship highlights that when the method converges, it does so with immense speed compared to linear techniques.

While originally designed for real-valued functions, extending Newton's method into the complex plane reveals a remarkably intricate dynamic. By mapping which complex initial guesses z_0 from Complex plane converge to a roots of a complex polynomial, the boundaries between these regions of attraction break down into self-similar, infinitely complex structures known as Newton fractals. These visual patterns beautifully show how simple mathematical iteration rules can give rise to highly sensitive geometric boundaries.

4.1 Python Implementation

```python
def newton(f, df, x0, eps, itmax):

    itnum = 1
    while True:
        denom = df(x0)
        if denom == 0:
            return None, 2

        x1 = x0 - f(x0) / denom

        if abs(x1 - x0) <= eps:
            return x1, 0

        if itnum == itmax:
            return x1, 1

        itnum += 1
        x0 = x1
f  = lambda x: x**6-x- 1
df = lambda x: 6*x**5-1

root, ier = newton(f, df, x0=2.0, eps=1e-7, itmax=100)

if ier == 0:
    print(f"Converged! Root = {root}")
elif ier == 1:
    print(f"Max iterations reached. Best estimate = {root}")
elif ier == 2:
    print("Error: derivative was zero at starting point.")
```

Listing 7: Newton's Method

5 Newton Fractals

When we apply Newton's method to a complex polynomial we come to see beautiful and colorful fractals. If you color each starting point in the complex plane based on the specific root it eventually converges to, you get an unexpected, infinitely detailed fractal structure. **Basin of attraction** Every root has a basin of attraction which is a region of the complex plane where any starting point inside it will converge to that root.

5.1 Newton's Method in the Complex Plane

Extend Newton's iteration to $\mathbb{C}$. For a polynomial $p(z)$:

$$z_{n+1} = z_n - \frac{p(z_n)}{p'(z_n)}, \quad z_n \in \mathbb{C}$$

For each starting point z_0 on a grid:

1. Run Newton's Method until convergence.
2. Record which root it converges to and the number of iterations taken to reach it.
3. Color the pixel at z_0 with that root's assigned color, using a different shade to reflect the iteration count.

The result is a **Newton Fractal**.

5.2 Example: $p(z) = z^3 - 1$

The three roots are:

$$z_1^* = 1, \quad z_2^* = e^{2\pi i/3} = -\tfrac{1}{2} + \tfrac{\sqrt{3}}{2}i, \quad z_3^* = e^{4\pi i/3} = -\tfrac{1}{2} - \tfrac{\sqrt{3}}{2}i$$

Newton's iteration becomes $z_{n+1} = z_n - \dfrac{z_n^3 - 1}{3z_n^2}$. Applied to a 1000×1000 grid over $[-2, 2] \times [-2, 2]$, the three colored basins of attraction meet at a boundary of infinite intricacy.

5.3 Python Implementation

```
import math
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import numpy as np

class Complex:
```

```
    def __init__(self, real=0.0, imag=0.0):
        self.real = float(real)
        self.imag = float(imag)
    def __add__(self, other):
        return Complex(self.real + other.real, self.imag + other.imag
    )
    def __sub__(self, other):
        return Complex(self.real - other.real, self.imag - other.imag
    )
    def __mul__(self, other):
        r = self.real * other.real - self.imag * other.imag
        i = self.real * other.imag + self.imag * other.real
        return Complex(r, i)
    def __truediv__(self, other):
        denom = other.real**2 + other.imag**2
        if denom == 0:
            raise ZeroDivisionError()
        r = (self.real * other.real + self.imag * other.imag) / denom
        i = (self.imag * other.real - self.real * other.imag) / denom
        return Complex(r, i)
    def __abs__(self):
        return math.sqrt(self.real**2 + self.imag**2)

def complex_pow(z, power):
    if power == 0:
        return Complex(1.0, 0.0)
    result = z
    for _ in range(1, power):
        result = result * z
    return result

def newton(f, df, x0, eps=1e-10, itmax=60):
    for step in range(itmax):
        fx  = f(x0)
        dfx = df(x0)
        if abs(dfx) == 0:
            return None, itmax
        x1 = x0 - fx / dfx
        if abs(x1 - x0) < eps:
            return x1, step
        x0 = x1
    return x0, itmax
n = 8 #no. of roots
def f(z):
    return complex_pow(z, 8) - Complex(1.0, 0.0) #define f

def df(z):
    return Complex(8.0, 0.0) * complex_pow(z, 7)#define df

np_roots = np.roots([1, 0, 0, 0, 0, 0, 0, 0, -1])
roots = [Complex(r.real, r.imag) for r in np_roots]
```

```
56
57 def get_root_index(root, tol=1e-2):
58     if root is None:
59         return -1
60     for i, r in enumerate(roots):
61         if abs(root - r) < tol:
62             return i
63     return -1
64
65 def compute_fractal(xmin, xmax, ymin, ymax, width, height, itmax):
66     root_grid  = np.zeros((height, width), dtype=int)
67     steps_grid = np.zeros((height, width), dtype=float)
68     for row in range(height):
69         for col in range(width):
70             real = xmin + (xmax - xmin) * col / width
71             imag = ymin + (ymax - ymin) * row / height
72             z0 = Complex(real, imag)
73             root, steps = newton(f, df, z0, itmax=itmax)
74             idx = get_root_index(root)
75             root_grid[row, col] = idx
76             steps_grid[row, col] = steps
77     return root_grid, steps_grid
78
79 ITMAX = 50
80 xmin, xmax, ymin, ymax = -1.5, 1.5, -1.5, 1.5
81 root_grid, steps_grid = compute_fractal(xmin, xmax, ymin, ymax, width
       =800, height=800, itmax=ITMAX)
82
83 rootColors = cm.rainbow(np.linspace(0, 1, n))[:, :3]
84 height, width = root_grid.shape
85 img = np.zeros((height, width, 3))
86
87 for row in range(height):
88     for col in range(width):
89         root_idx = root_grid[row, col]
90         steps    = steps_grid[row, col]
91         if root_idx == -1:
92             img[row, col] = [0.0, 0.0, 0.0]
93         else:
94             base_color = rootColors[root_idx]
95             norm_steps = steps / ITMAX
96             grad_factor = max(0.35, 1.0 - (norm_steps ** 0.6))
97             img[row, col] = base_color * grad_factor
98
99 plt.figure(figsize=(10, 10))
100 plt.imshow(img, extent=[xmin, xmax, ymin, ymax], origin='lower',
       interpolation='bilinear')
101 plt.xlabel('Re(z)')
102 plt.ylabel('Im(z)')
103 plt.tight_layout()
104 plt.show()
```

Listing 8: Newton Fractal Generator of $z^8 - 1$

5.4 Gallery of Newton Fractals

- $p(z) = z^3 - 1$: Three-fold symmetry, muted blues, reds, greens
- $p(z) = z^4 - 1$: Four-fold symmetry, spiraling tendrils
- $p(z) = z^5 - 1$: Five quadrants with intricate cross-boundary detail
- $p(z) = z^8 - 1$: Eight-fold symmetry, extraordinarily fine structure

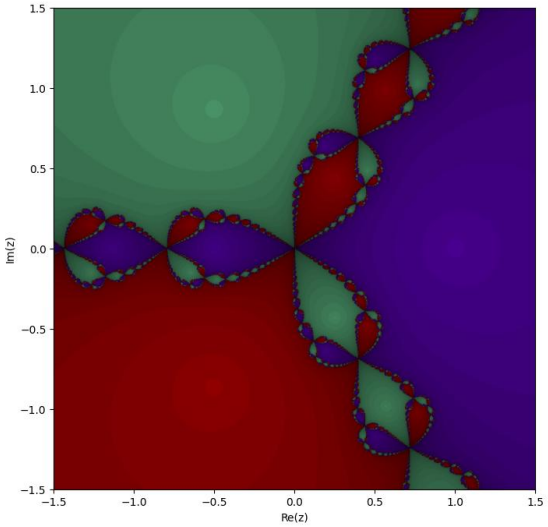

Figure 3: Fractal: $p(z) = z^3 - 1$

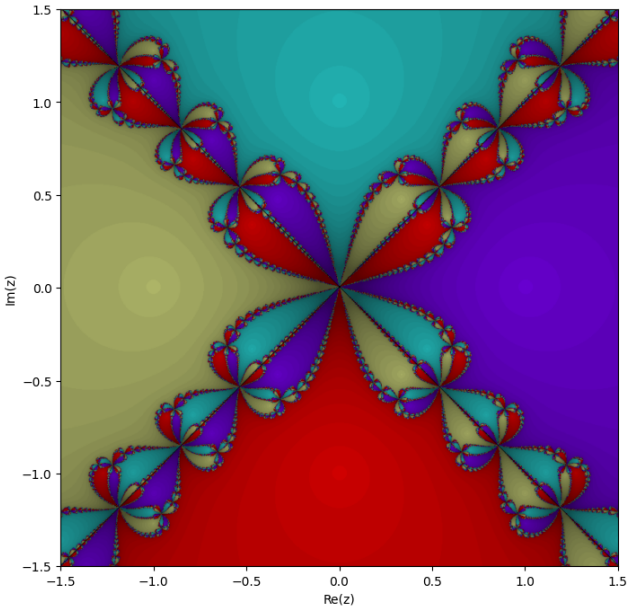

Figure 4: Fractal: $p(z) = z^4 - 1$

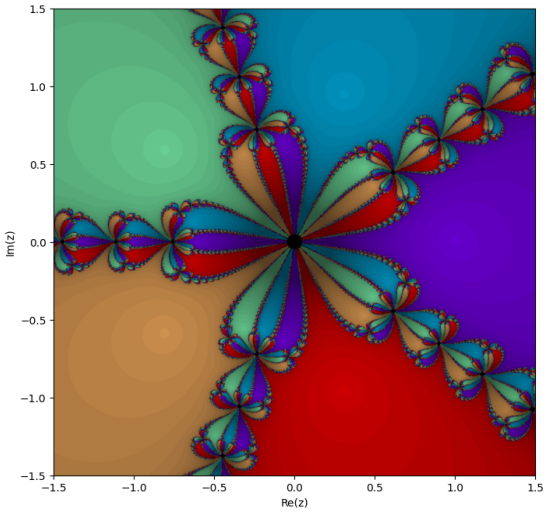

Figure 5: Fractal: $p(z) = z^5 - 1$

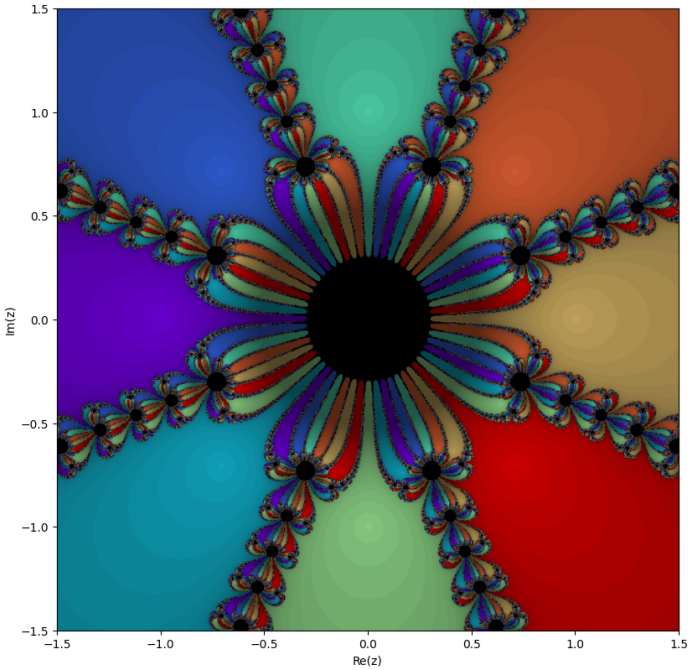

Figure 6: Fractal: $p(z) = z^8 - 1$

6 Conclusion

This paper reviewed classical numerical algorithms, connecting elementary root-finding methods to the complex geometry of Newton fractals.

Each root-finding method presents distinct property of convergence. The **Bisection Method** guarantees convergence at the expense of linear speed, making it highly reliable when stability is prioritized over performance.
The**Secant Method** provides superlinear convergence without requiring derivative evaluations, though poor initialization risks divergence.
Newton's Method delivers rapid quadratic convergence near simple roots, standing out for its efficiency under proper starting conditions. If the initial guess is too far from the root it does not converge.

For polynomial interpolation, Lagrange and Newton's Divided Difference methods generate the exact same unique polynomial through $n+1$ data points. However, Newton's framework is practically advantageous when data points are added incrementally. Cubic Hermite Interpolation extends these concepts by matching both function values and first derivatives at the nodes.

Finally, exploring Newton Fractals demonstrates how standard numerical algorithms can yield complex mathematical imagery. When Newton's iteration is evaluated in the complex plane, the boundaries between convergence basins forming fractal structures. This sensitivity to initial conditions highlights a deep geometric property rather than a mere computational error.

Future research will focus on developing interactive fractal visualizations with real-time zoom capabilities, extending these approaches to systems of nonlinear equations via the multivariate Newton's Method, and conducting a formal performance comparison of convergence rates across all discussed methods.

References

[1] Wikipedia contributors. Newton fractal. *Wikipedia, The Free Encyclopedia*, 2024.

[2] Atkinson, K. E. *An Introduction to Numerical Analysis*, 2nd ed. John Wiley & Sons, New York, 1989.

[3] Trefethen, L. N. and Bau, D. *Numerical Linear Algebra*. SIAM, Philadelphia, PA, 1997.